FTQC Resource Estimator

Research interview and validation addendum

Scope: independent source-code memory simulation, statistical resource feasibility and explicitly hypothetical controller scheduling. Supplement to the earlier eight-page code-aligned study guide.

What was actually established?

The owner executed the original nine-round surface-code memory simulation on a Mac (180,000 trials at distances 3, 5, 7 across three p settings) and supplied a separately seeded d=7/9 follow-up (40,000 trials). The fixed-nine-round d=9 observation had 10 failures among 20,000 shots at p=0.003; its upper 95% Wilson endpoint was about 0.092%. Under 40 independent memory events and a 10% workload-failure target, the per-event budget is approximately 0.263%. This passes the modeled conservative criterion.

The initially infeasible d=3/5/7 sweep was scientifically useful; it motivated the distance-9 experiment. This is a simulated memory conclusion, NOT a fault-tolerant gate, physical quantum processor benchmark, or hardware-specific architecture guarantee.

implementation and validation status

New: a literature-matched rounds=distance benchmark (code and tests implemented, real backend execution pending in the packaging environment); a distinct readout-channel scheduling/power envelope with separate physical-qubit accounting; tighter full-publication evidence gates. The benchmark must execute with real Stim and PyMatching on the owner's machine, generate all 20 observations and replay its exact seeds and circuit hashes before its figure is publishable.

No new decoder or new quantum error-correcting code is claimed. The research contribution is the independently developed integration of simulation evidence, uncertainty, protocol sensitivity and architecture selection under explicit constraints.

1. Reliability-constrained memory design

Derive the independent-events budget

If a workload uses N=4 patches across W=10 independent memory windows, there are E=N×W=40 independent events. To keep total failure at or below F=0.10, the maximum per-window memory failure probability is q = 1 – (1 – F)^(1/E) = 0.00263055.

For each code distance d, Stim samples a memory experiment and PyMatching produces an MWPM prediction. The estimated logical failure fraction is failures/shots. A 95% Wilson binomial interval quantifies statistical uncertainty on one point. The model labels a design conservatively feasible only when the **upper Wilson endpoint** is at or below q.

Caution: separately calculated Wilson 95% bounds are not simultaneous 95% bounds for the entire experiment family; multiplying their endpoints across memory windows additionally assumes independent events.

The distance-9 follow-up

For the owner-supplied d=9, nine-round X-memory data at p=0.003: failures=10, shots=20,000. The observed failure fraction is 0.0005 (0.05%). Upper Wilson 95% is about 0.00092 (0.092%), less than the 0.00263 budget. The d=7 follow-up had 57/20,000 (0.285%) and did not meet the upper-bound target.

Physical patch qubits in the simple rotated-code layout are $2d^2-1$. Across four patches: d=7 requires 4×97=388 data/ancilla qubits, while d=9 requires 4×161=644. This 66% count increase buys a meaningful modeled reduction in memory failure under the specified noise. There is no claim that those physical qubits include controllers or that a physical machine implements this schedule.

Interview challenge

Can you explain why the original feasibility frontier was empty and why increasing code distance changed the reliability result? Why would simply increasing the number of controller channels not improve a fixed Stim error probability? Distinguish the model's robust criterion from the observed point-estimate criterion.

2. A published-reference-matched stress test

Reference and method

The Riverlane/Deltakit public textbook chapter *Simulating surface codes using stim* demonstrates rotated X-basis surface-memory circuits in Stim, four equal circuit noise knobs, PyMatching decoding via the decomposed detector error model, and **rounds=d**. It shows qualitative curve crossings near p≈0.007 under its reported sweep. Our implementation matches the *protocol*, but not its much denser noise sweep or per-point sampling count.

Reference: https://textbook.riverlane.com/en/latest/notebooks/ch5-decoding-surfcodes/simulating-surface-codes-stim.html

Our registered suite proposes distances {3,5,7,9}; p={0,.003,.005,.007,.009}; 20,000 shots per point; X memory; rounds=d. That is 20 points or 400,000 total shots. A noiseless control is included at each distance. The runner records generated circuit hashes, seeds, software versions, sample counts, MWPM disagreements and replay evidence.

Protocol comparison: fixed nine rounds versus rounds=d

Code distance	Existing experiment	Literature-aligned experiment
d=3	9 syndrome rounds	3 syndrome rounds
d=5	9 syndrome rounds	5 syndrome rounds
d=7	9 syndrome rounds	7 syndrome rounds
d=9	9 syndrome rounds	9 syndrome rounds

Different d values in the literature benchmark have different memory durations. A smaller observed pL at larger d is informative about error suppression, but is **not** a like-for-like equal-duration test. Do not pool or directly compare the two protocols without saying what changed.

Evidence boundary

The published textbook is an educational technical reference, not an original peer-reviewed benchmark figure that we have reproduced digit-for-digit. The original Stim simulator paper is Craig Gidney, *Quantum* 5, 497 (2021). Our subset is a method validation and qualitative comparison, not a fitted physical threshold or an original numerical reproduction of that paper.

3. Controller capacity, realism and novelty

Separate quantities with real units

For d, N independent rotated patches require $N(2d^2-1)$ simple data-and-ancilla physical-qubit equivalents. Per round, $N(d^2-1)$ stabilizer measurements are counted. If C readout channels can be used in parallel, an intentionally simplified batch envelope is $B = \text{ceil}[N(d^2-1)/C]$. Its nominal per-round duration is max(base-cycle-time, B×slot-time). The model also includes a fixed decoder response overhead per window.

The output separates: physical patch qubits (count), readout channels (count), assumed controller power (W) and schedule time (microseconds). It does not add a classical readout channel or a watt to a physical qubit. The current numerical inputs are hypotheses and are NOT manufacturer specifications.

Why this is still not hardware calibrated

An executable stabilizer-measurement sequence imposes gate, connectivity and reset constraints that this envelope ignores. Additional waiting may introduce idle faults and change the logical error distribution, but the current Stim circuit is unchanged when scheduling is slowed. Meaningful hardware evaluation needs calibrated error-versus-time, readout multiplexing, crosstalk/leakage, decoder throughput and device topology, and then fresh Stim circuits (or other simulators where non-Pauli effects matter).

Research novelty: accurate positioning

Current original integration: a reliability-budget decision workflow that separates simulation evidence, sampling uncertainty, protocol-matching checks, and physical/control cost dimensions. This is a credible research-internship project. It is **not** a claim of a novel matching decoder, new fault-tolerant architecture, or previously unknown scientific theorem. A genuinely original follow-up hypothesis is that calibrated readout-induced delays shift the reliability-feasible distance/controller frontier. That hypothesis is **planned, not tested**.

Researcher interview questions

1. When does using marginal upper Wilson bounds misstate joint confidence? 2. What changes when syndrome rounds scale with distance? 3. Which quantities are true patch counts and which are modeled electronics? 4. What experiments would falsify the assumption that controller timing leaves logical noise unchanged? 5. Why does a toy non-dominated frontier fail to establish superiority on real hardware?

4. Local release and personal sign-off

On your Mac (real scientific backends installed)

```
mkdir -p logs

./RUN_STIM_MEMORY_FOLLOWUP.sh

./RUN_LITERATURE_BENCHMARK.sh

./RUN_CONTROLLER_CAPACITY.sh

./RUN_ALL_VALIDATIONS.sh

python3 scripts/prepare_results.py --mode full --output-dir build/publication
```

The complete pipeline re-runs previous baseline and follow-up experiments and the new 400,000-shot benchmark; the full publication bundle is designed to fail if the required real-backend replay certificate is missing. Generated results live under outputs/ and the safe copy under build/publication. Nothing is pushed to GitHub by these local commands.

What you should verify yourself

Check all 20 benchmark observations, the zero-noise controls, software versions, replay hashes and per-point intervals. Compare the error-suppression direction with the reference chapter; if your crossover differs, record it and evaluate sampling, software-version and protocol effects. Review the controller preview as hypothetical: physical qubits and electronics numbers must not be summed. Reconcile every README claim with its artifact and validation certificate.

Final scientific status

Earlier owner runs: real and supplied. New literature benchmark: code/test ready, NOT executed in the package-building environment. controller capacity: offline preview only from prior owner-supplied data, NOT physical calibration. No code here claims a new quantum algorithm or peer-reviewed paper figure reproduction.